

Facial Mask Detection Project Report

Authors:
Oumaima Abid
Sofiane Lahouar
Zeineb Ghorbel

Pedagogic Advisor:
Mr. Riadh Tebourbi

Acadamic Year: 2019/2020

SUP'COM
Higher School of Communication of Tunis

Contents

General introduction 2
Chapter 1: Business understanding of the project 3
Identifying the problem 3
The proposed solution 4
Chapter 2: Project pipeline 5
Introduction 5
From a High-level ML Pipeline to a computer vision pipeline 5
Phase 1: Data understanding 6
Phase 2: Data preprocessing 7
Phase 3: Modeling 9
Phase 4: Training and validation 12
Phase 5: Testing 13
Conclusion 15
Chapter 3: Experimentation and fine-tuning 16
Introduction 16
Strategy 1: Changing the learning rate 16
Strategy 2: Changing the Batch size 18
Strategy 3: changing the number of hidden layers and neurons 19
Evaluating the result 21
General Conclusion and Perspectives 23
Bibliography and Webography 24

Table of figures

Figure 1: the efficacy of wearing facial mask in fighting the covid-19 outbreak 3
Figure 2: a simple flowchart of the proposed solution 4
Figure 3: CRISP-DM Workflow 5
Figure 4: Examples of cropped faces 8
Figure 5: a visualization of the splits 8
Figure 6: ResNet50 Architecture 9
Figure 7: a simple representation of a neurons and a sequence of layers 10
Figure 8: SGD in execution 11
Figure 9: Forward and backward propagation 12
Figure 10: forward propagation on validation dataset 13
Figure 11: Selection of the best model 13
Figure 12: Examples of the find result 14
Figure 13: the effect of learning rate 16
Figure 14: comparison of different learning rates 17
Figure 15: dynamic learning rate and its effect 17
Figure 16: different batch sizes and its effect on the training process 18
Figure 17: General structure of the confusion matrix 21

General introduction

The catastrophic outbreak of Coronavirus also known as COVID-2019 has brought the worldwide threat to the living society. The whole world is putting incredible efforts to fight against the spread of this deadly disease in terms of infrastructure, finance, data sources, protective gears, life-risk treatments and several other resources. The artificial intelligence researchers are focusing their expertise knowledge to develop mathematical models for analyzing this epidemic situation using nationwide shared data. With the Covid-19 health crisis, the wearing of masks has become the norm in many gathering places: shops, transport, etc.

Indeed, many public places require their visitors to wear a mask in order to protect the greatest number. we are developing a solution to detect the wearing (or not) of the mask within a crowd of people in public places.

How to achieve such an accurate detection? Thanks to artificial intelligence (AI), our image analysis software has been specially trained to identify a mask on a human face. If the position of the camera allows, detection becomes possible.

In this document, we will start with identifying the problem and the required solution. Next, we will divide into each phase on how we turned a solution on paper into a real working prototype. And finally, we will discuss all the approaches we've done in order to improve the performance of our prototype.

Chapter 1: Business understanding of the project

Identifying the problem

In the figure below, an interesting study by Jeremy Howard from the university of San Francisco shows the importance of wearing facial masks to reduce the Covid-19 outbreak. If more people wear masks and wear them correctly, we will be in the blue zone with a reproduction number below 1. It means that the virus will spread slowly, and if sustained, it will end the outbreak.

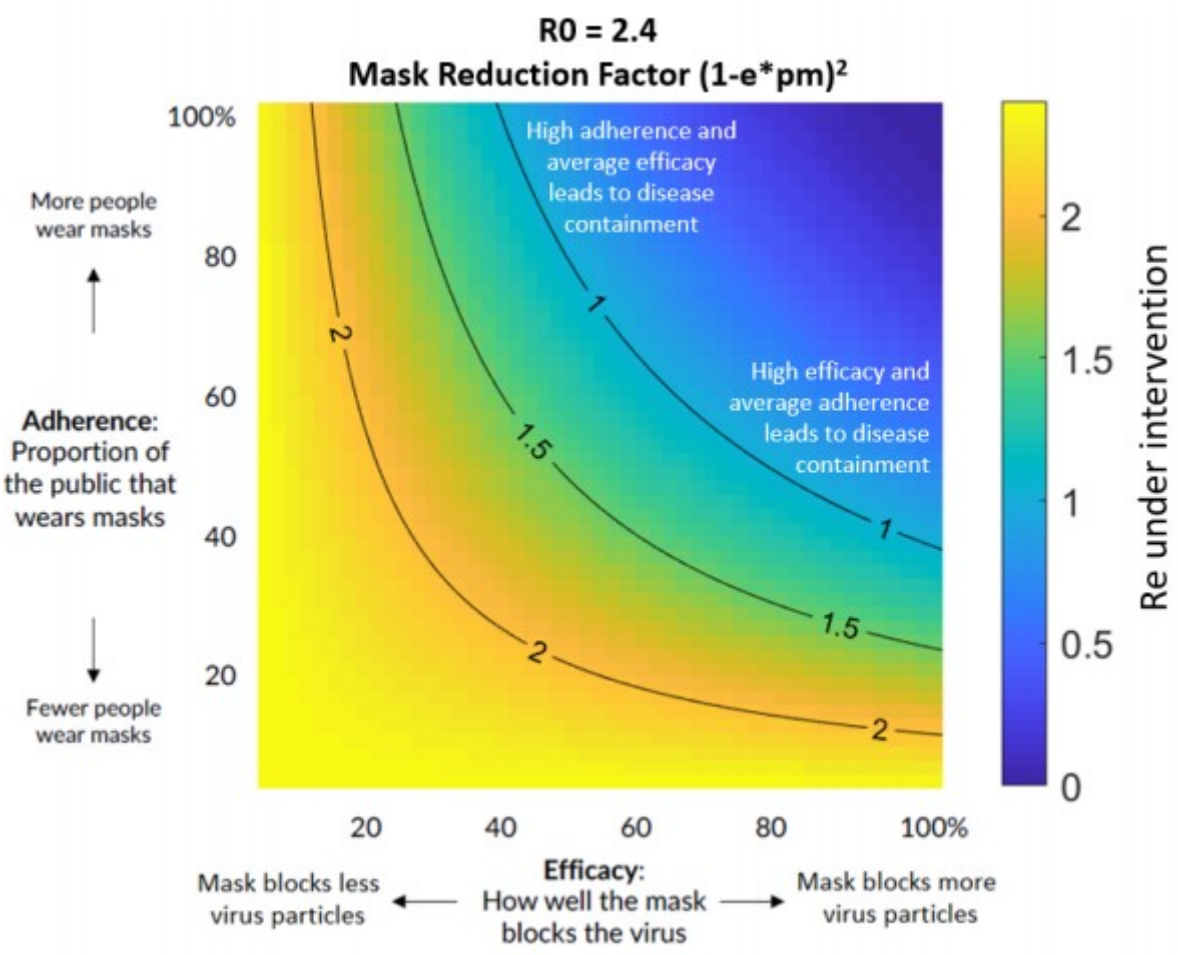


Figure 1: the efficacy of wearing facial mask in fighting the covid-19 outbreak

Therefore, after initial debate, the utility of masks during the Covid-19 pandemic seems widely accepted now. The dominant scientific opinion says masks are very useful, and even relatively simple home-made masks can offer a great degree of protection against the novel coronavirus. With several countries like Singapore, Japan, states, and cities in the United States like Texas

and New York, wearing masks is becoming mandatory for the public. That's why the need for an automated system that is capable of helping authorities maintain guidelines is important.

So, our problem is: how do we make sure that people are respecting the government guidelines about wearing facial masks in crucial places like hospitals, post offices, etc.?

The proposed solution

To limit the virus spread, an increased surveillance must be implemented without using many humans because it can risk in spreading the virus. This is where a machine learning solution comes in place. The solution consists of building an intelligent monitoring system that can classify if a person is wearing a mask or not. This system will be installed in the monitoring room of a building utilizing the cameras inside and outside the building, and it will alert employee if there are people not following the guidelines. This document is focused on how to implement the classification part. Further information about deployment and integrating it in a real monitoring system will be discussed in a future document.

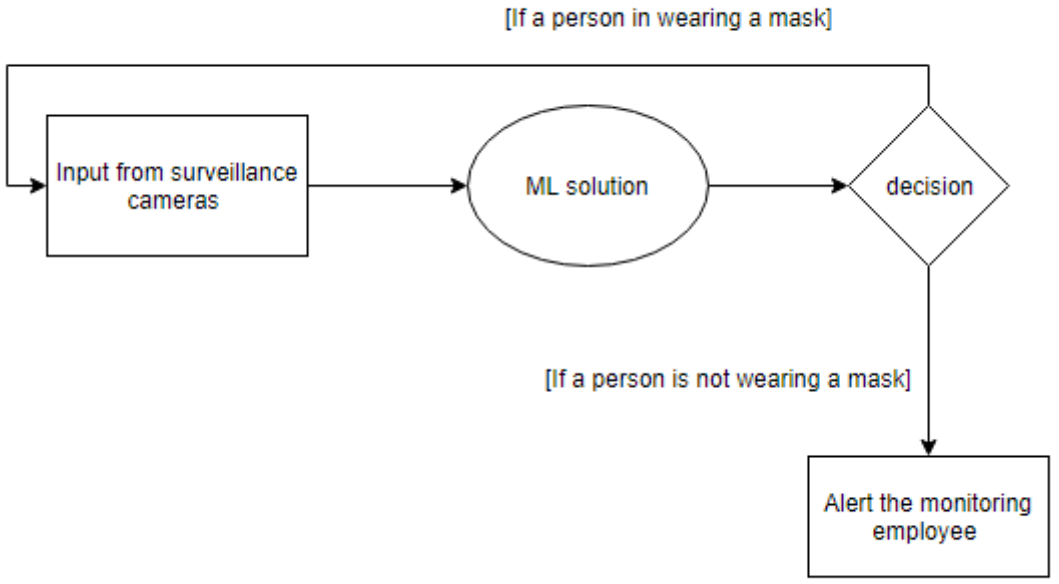


Figure 2: a simple flowchart of the proposed solution

Chapter 2: Project pipeline

Introduction

Defining a pipeline is way of identifying the stages of a project with a checklist for each stage. In this chapter, we will define our project's pipeline and detail each one of its stages.

From a High-level ML Pipeline to a computer vision pipeline

One of the most used processes for machine learning projects is Cross Industry Standard Process for Data Mining (CRISP-DM). CRISP-DM outlines the needed steps and pipeline for achieving a successful ML project, from business understanding to the deployment stages and all the stages in between. This model is divided into six iterative stages that revolves around the data.

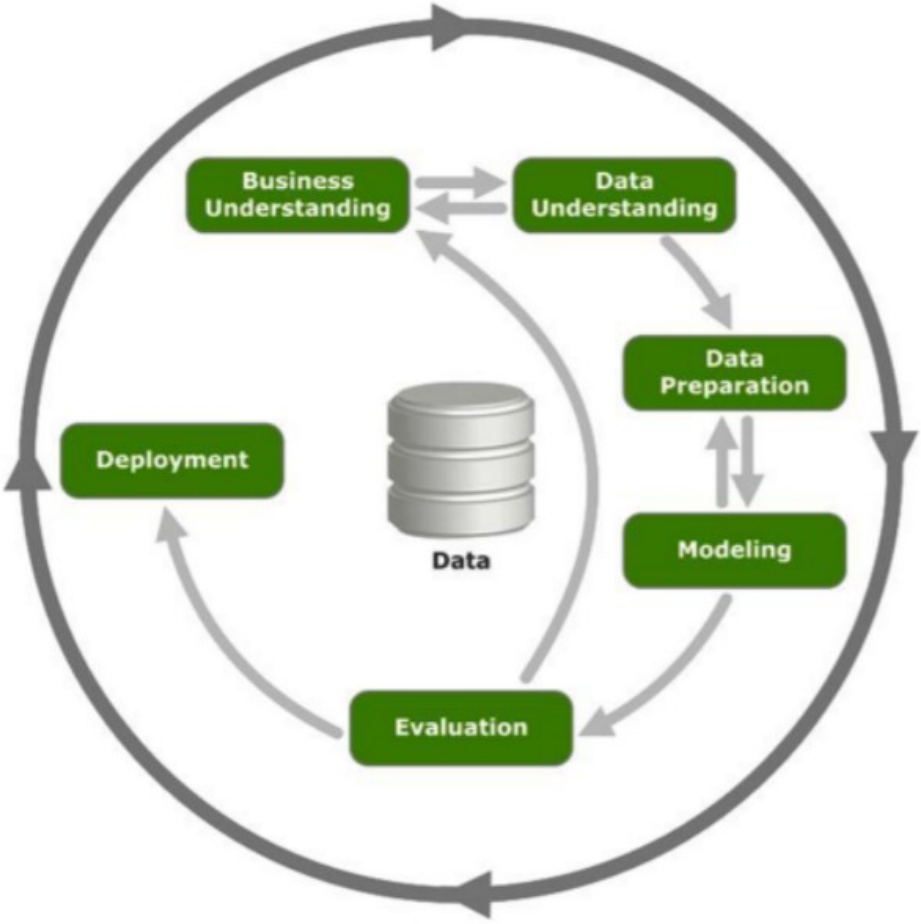


Figure 3: CRISP-DM Workflow

Business Understanding is the first and most crucial stage to begin with. It helps in defining the business requirements and setting expectations and success criteria of the project. For our project:

- Be able to distinguish between people wearing facial masks and people who aren't wearing facial masks through a real-time feed camera.
- Achieve a success rate over 80%
- Achieve a high framerate (> 20 fps)
- Deploy the project in a real environment

Data understanding is the art of collecting and understand the data that will satisfy the business understanding.

Data preparation or Data preprocessing is the most time-consuming step in any machine learning project. It consists of data integration, data imputation, data formatting, feature selection, and feature extraction.

We will see in the next section how feature selection and feature extraction will be translated to the modeling step.

Modeling is where the machine learning/deep learning algorithms and models come in place. It involves choosing the right tools and frameworks and building the model.

Evaluation is where we test our model and see if the results satisfies the criteria that we defined in the first step.

Deployment is the final step in which we put the model in a real environment and monitor the results for future improvements.

Now, the only difference between the CRISP-DM pipeline and our computer vision project pipeline is that the second section of data preparation (feature selection and feature extraction) is moved to the modeling stage which we will explain why in the next sections.

Phase 1: Data understanding

After hours of searching, we found 3 potential datasets that will be excellent for our project case.

- Medical Masks Dataset from Kaggle.com (https://www.kaggle.com/vtech6/medical-masks-dataset) : it contains 682 images with over 3000 faces wearing masks and around 700 faces without masks. It is well structured meaning that each image comes with a label file (.xml) containing coordinates of a face in an image, and if the face has a mask or not.

- Mask Wearing Dataset from Roboflow.ai (https://public.roboflow.ai/object-detection/mask-wearing/) : it contains 149 images with 806 faces wearing masks and 148 faces without one. Also, it is semi structured because each image comes with a label file (.xml) containing the placement of a face in an image, but it doesn't tell if it's wearing a mask or not.
- Spot the mask challenge by Zindi (https://zindi.africa/hackathons/spot-the-mask-challenge) : it contains 1800 images and comes with a label file (.csv) saying if the image contains people wearing masks (1) or not (0) but it doesn't come with the location of faces in the picture.

Decision Taken: We decided to use the medical masks dataset from Kaggle to train and test the model for being the most structured one. For future work, we will use the other 2 datasets to evaluate and tweak the model in order to reach better results.

Phase 2: Data preprocessing

In order to understand what kind of preprocessing we need to do; we need to rewrite the business statement of our problem in a more technical way and identify key words.

"Build and train a model that will classify faces into 2 classes: wearing a mask or not wearing a mask"

Let's start with the keyword "faces". it means that only the faces are needed it to achieve the task and not the whole image. Therefore, we will need to crop the faces from the images using the label file provided with each image since it has the location of each face (xmin,xmax,ymin,ymax).

Next, we have "2 classes". It indicates that we need to separate the faces into 2 folders (0 for wearing a mask and 1 for not wearing a mask).

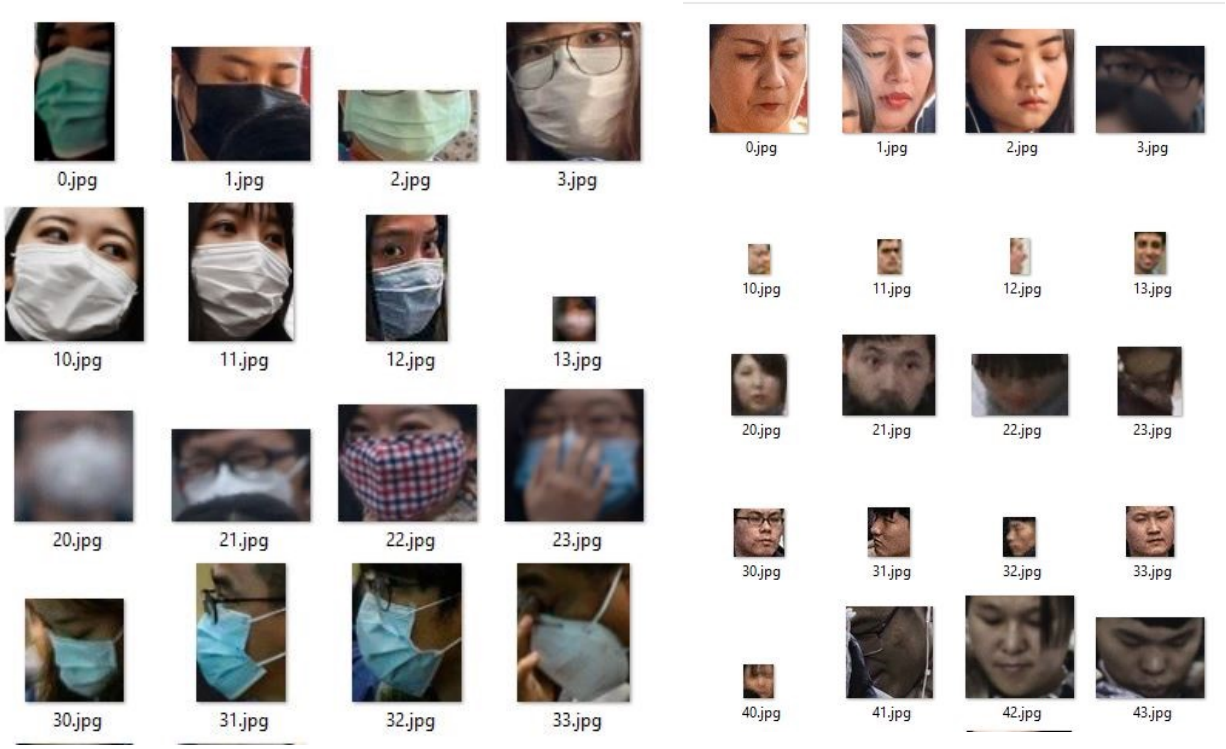


Figure 4: Examples of cropped faces

Now, for the keyword "train", training a model means we need 3 subsets of datasets:

- Training dataset: used for training and updating the weights of our model with the objective to reduce the training loss function in each iteration (epoch)
- Validation dataset: used to select which model to save from the total epochs by selecting the one with the smallest validation loss
- Testing dataset: used to test the accuracy of the model (More like validating the selected model again)

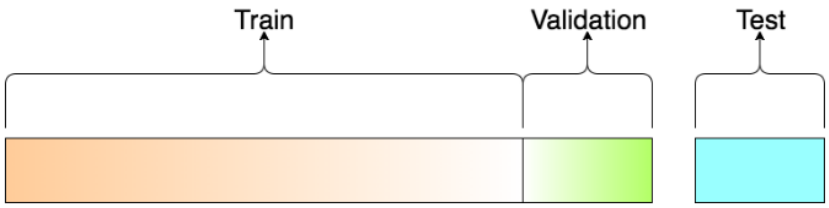


Figure 5: a visualization of the splits

So, the cropped labeled faces from the images need to be shuffled and split into the 3 datasets mentioned above.

Finally, the last keyword is "model". Since we will be using a CNN model as the first block of our model, the images need to be formatted (resized and normalized) into a certain format according to the chosen CNN model.

Phase 3: Modeling

To understand the model used in the project, we need to first understand the concept of Convolutional Neural Networks (CNN). CNNs are neural networks that utilize the convolutional operation to extract and select the best features in an image. It creates custom filters for these features in which they will be used later to classify the image. For example, detecting where a mask is worn (mouth and nose) by creating filter that will extract that area is a feature that can be used. Developing a CNN from scratch is a time-consuming task and require tons of computational power to train it. Fortunately, thanks to years of research and the work of hundreds of Machine Learning engineers, pretrained CNN models for features extraction and selection were made and one of them is the ResNet50 architecture. Residual Networks (ResNet) are like any other CNN architecture with n blocks of convolutional layers but with small change. It introduced the concept of "skip connection" which allows the ResNet architecture to skip training some layers if they are unnecessary. This technique reduces the computational time of training the model and improves its accuracy. It's why ResNet architecture won the biggest image classification competition in 2015 (ImageNet 2015) with a top-5 score of 3.57% (beyond human ability).

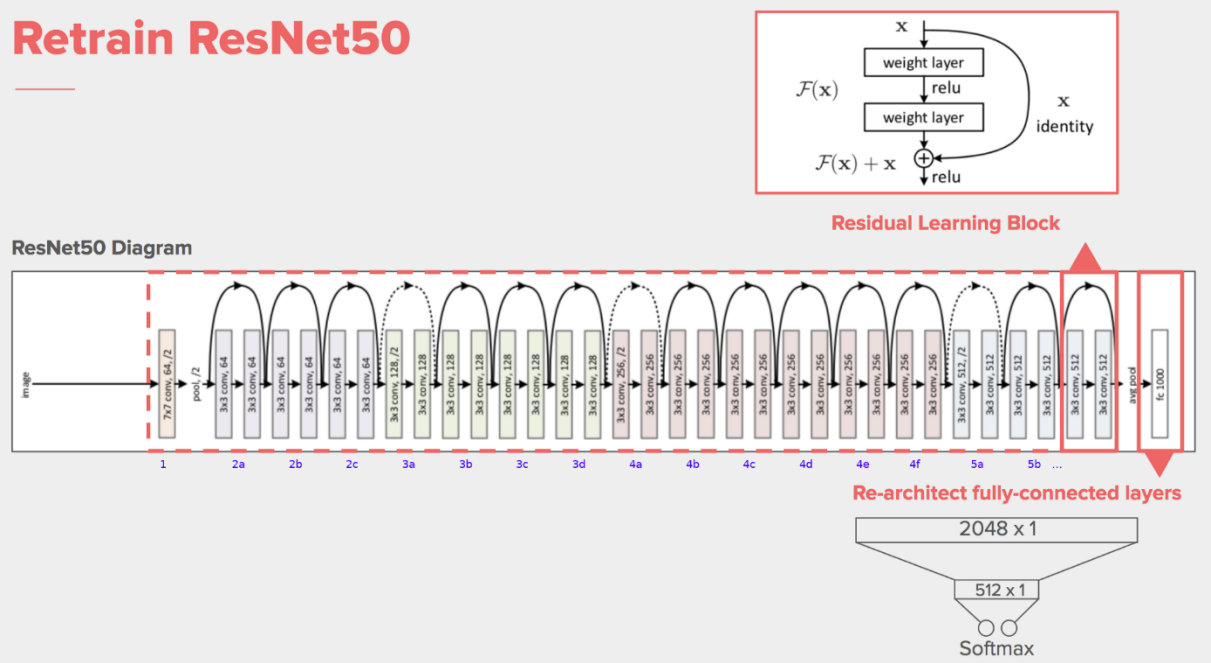


Figure 6: ResNet50 Architecture

ResNet is used to classify 1000 objects in images and thus it has a classification neural network with 1000 outputs. Thanks to the power of transfer learning, it's possible to take just the

convolution blocks (the features extractors and selectors) and add to it our own classification neural network with only 2 outputs (wearing a mask or not).

Before we tackle the subject of classification, we need to understand how neural networks are made. A neural network is a sequence of connected layers where each layer contains a set number of neurons. A neuron basically does 2 steps:

- Multiply each coming input with a certain weight, add the results together, and then add a bias
- Pass the result of the first step through an activation function to decide if the output has a meaningful value or not, hence the name "activation" function.

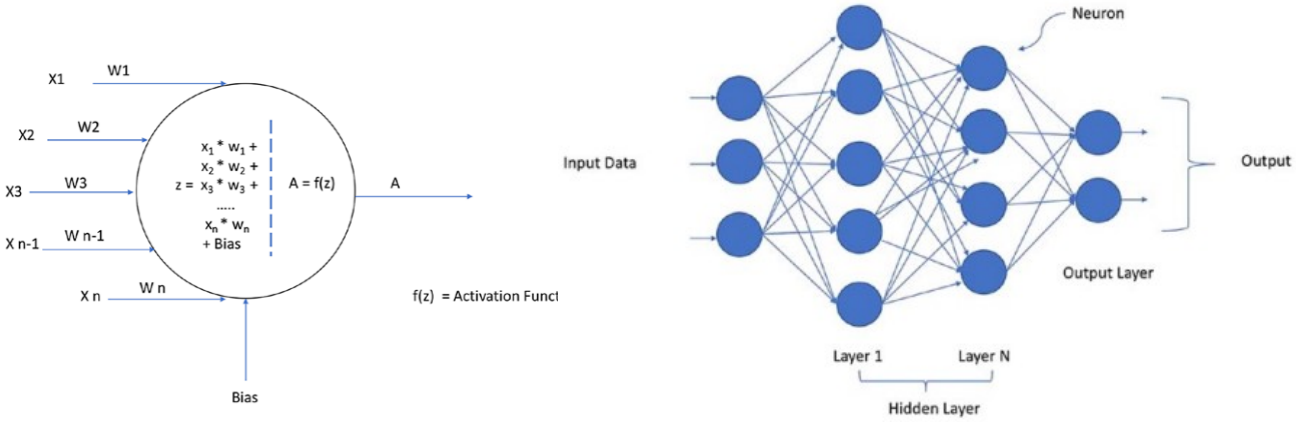


Figure 7: a simple representation of a neurons and a sequence of layers

The final Convolution block of the ResNet50 architecture has 2048 outputs, therefore, our classification neural network will have 2048 neurons in the first layer and 2 neurons in the final layer. Choosing how many layers or how many neurons in each layer between the first and last layers is a subject that still in discussion today. Many have conducted that trial and error is the best approach. It means there is no certain formula or rule to follow, we need to set and modify the inside layers accordingly to the accuracy of our model. Although what we can make a decision on are the activation functions. For the first and hidden layers, the ReLU (Rectified Linear Unit) function has proven to be computationally efficient compared to the other two popular activation functions Sigmoid and Tanh.

$$ReLU: f(X) = \max(0, X), Sigmoid: f(X) = \frac{1}{1+e^{-X}}, Tanh: f(X) = \tanh(X) = \frac{e^{X} - e^{-X}}{e^{X} + e^{-X}}$$

We can conclude from the equations above that for high value of X, there is almost no change to the output of the activation function, thus it will confuse the network and make it too slow to reach a correct predication. But, for the last layer (the output layer), we will be using a SoftMax activation function that gives an output between 0 and 1. This is useful because the output will be in a form of probability, and it will be easy to decide the correct prediction (the one with the highest probability).

$$Softmax: S(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

Now before we move to the next phase, 2 important parameters must be defined; the loss function and the learning rate. The goal of a neural network is to find the global minimum of a loss function through optimization. One of the most used optimization algorithms in neural networks is the Stochastic Gradient Descent (SGD).

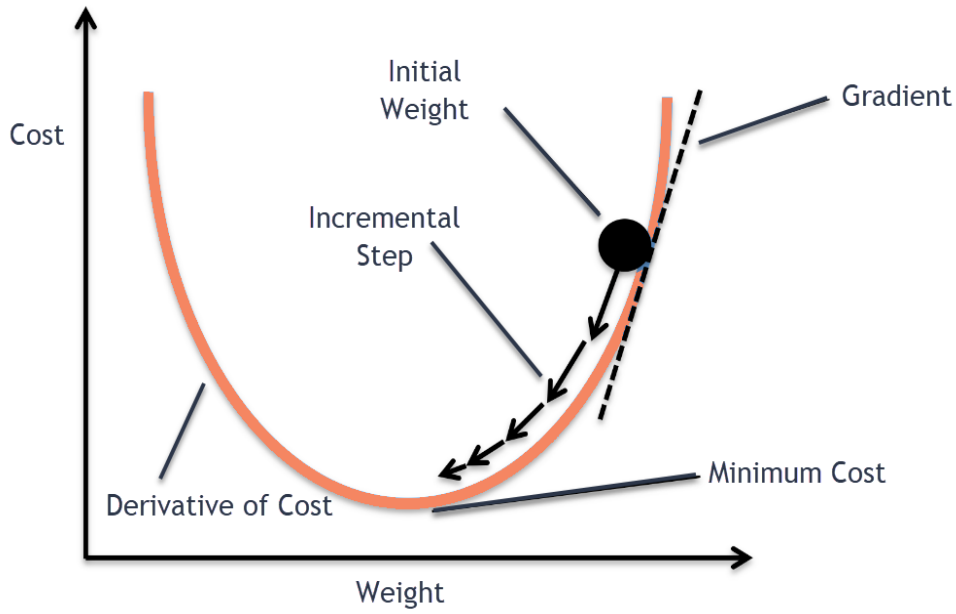


Figure 8: SGD in execution

It calculates the gradient of the loss function to give us and estimate of the weights update that will help in reducing the value of loss function. This gradient will be controlled by the learning rate meaning that we can decide the length of the step taken by the SGD algorithm to reach the global minimum. Choosing the right value of learning rate can prevent the minimum skipping problem (a step so large that it will go over the global minimum).

$$Weight_i = Weight_i - LearningRate * \frac{\partial LossFunction}{\partial Weight_i}$$

For the loss function, the cross-entropy loss function is a go-to for classification problems because it prevents the vanishing gradient problem meaning for large values of inputs will imply small changes to the loss function, thus there will be no update to the weights and confuse the network.

$$Binary\ cross-entropy: Loss = (Y_{model})(-\log(Y_{pred})) + (1 - Y_{model})(-\log(1 - Y_{pred})$$

Phase 4: Training and validation

Training a neural network is very simple. First, we need to do a forward pass in which we feed the model with the data and calculate the loss function. Next, the backpropagation is preformed in which the gradient of loss function is calculated and then update the weights of the trainable parameters. This step is repeated for a specific number of iterations (epochs). Each epoch will produce a model that we can save and use in a production environment, but how do we know which one to choose?

```
#reset the gradient to zero
optimizer.zero_grad()
#Forward propagation
result = model(X) #feeding the data to the model
loss = criterion(result, y) #calculate the loss function
epoch_train_loss += loss.item()
#Backpropagation
loss.backward() #calulate the gradient of the loss function
optimizer.step() #update the weights
```

Figure 9: Forward and backward propagation

Like we said, the main goal of train is to reduce the loss function output, so we will see that reduction in each iteration. But, that doesn't mean that the model is preforming well, It can be overfitting. Overfitting means that the model learned the exact data characteristics, including the noise, and therefore it will fail to predict reliably on new unseen data.

This is where the validation step comes in hand. During each epoch we will feed the model data from the validation dataset without preforming optimization and calculate the validation loss. Then, we save the model with the lowest validation loss.

```
with torch.no_grad(): #without updating the weights
    for X, y in val_loader:

        X, y = X.cuda(), y.cuda()

        result = model(X) #feed the validation data to the model
        loss = criterion(result, y) #calculate the loss function
        epoch_val_loss += loss.item() #calulate the epoch val loss value
        _, maximum = torch.max(result.data, 1) #predict the output from the model
        total += y.size(0)
        correct += (maximum == y).sum().item() #calculate the correct predections

val_losses.append(epoch_val_loss) #Store the values of val loss of each epoch
accuracy = correct/total #caluclate the accuracy of the validation dataset prediction
```

Figure 10: forward propagation on validation dataset

```
if min(val_losses) == val_losses[-1]: #if the value of current val loss is lower than pervious values of val loss
    best_epoch = epoch #save the model created by this epoch
    checkpoint = {'model': model,
                  'state_dict': model.state_dict(),
                  'optimizer' : optimizer.state_dict()}

    torch.save(checkpoint, models_dir + '{}.pth'.format(epoch))
    print("Model saved")
```

Figure 11: Selection of the best model

Note: If the loss function value is going up or fluctuating, it means we have a problem and we need to preform some tweaking to the model.

Phase 5: Testing

For testing, we created 2 python scripts, one for detecting masks in images and one for a real-time video feed. The scripts have 2 roles:

- Detecting faces in the image(frame)
- Put a green rectangle on the face if the person is wearing a mask and a red rectangle otherwise

Some examples of the final result:

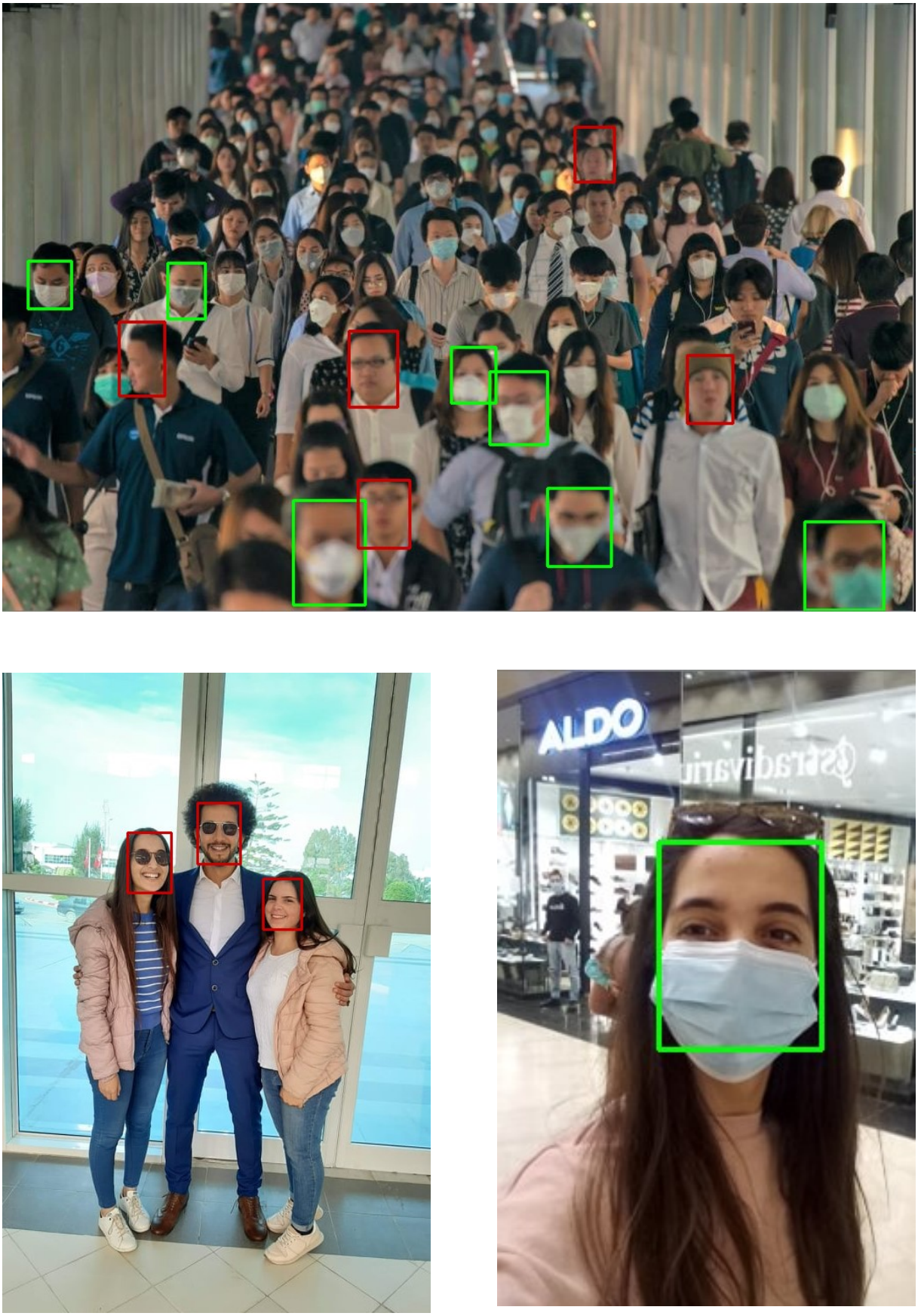


Figure 12: Examples of the find result

Conclusion

In summary, most of the time was spent in training the model, specifically in experimentation with different values and fine-tuning the model. We achieve a decent accuracy, and we had a great frame per second value (20 fps) on a slow laptop.

Chapter 3: Experimentation and fine-tuning

Introduction

Like we said before, it's all about trial and error. There is no a certain rule or formula to fix some parameters like the numbers of hidden layers and hidden neurons, the learning rate, and the number of iterations. In this chapter, we will discuss the different approaches used in order to improve the accuracy of our model and optimize its training.

Strategy 1: Changing the learning rate

The value of learning rate decides how much should the gradient move to get close to global minimum of the loss function. Setting too high will make it hard for the model to converge, and setting too low will make the training too slow.

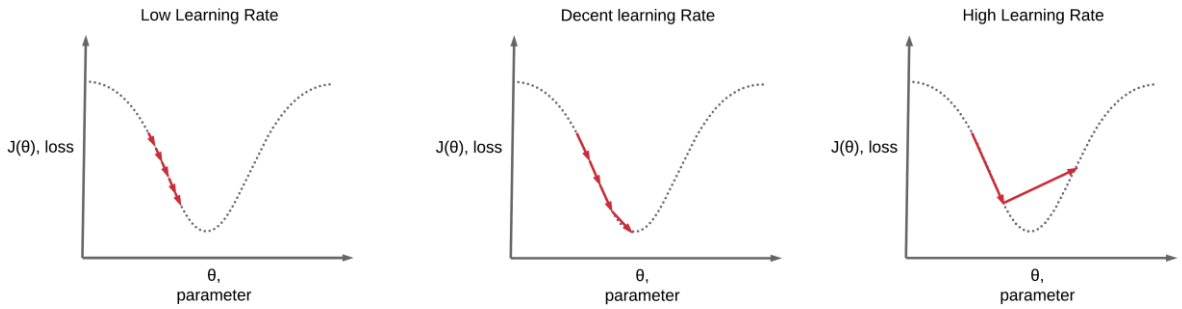


Figure 13: the effect of learning rate

So, a good approach is to start with a high reasonable value and start decreasing it after each training until reaching a decent accuracy with less time.

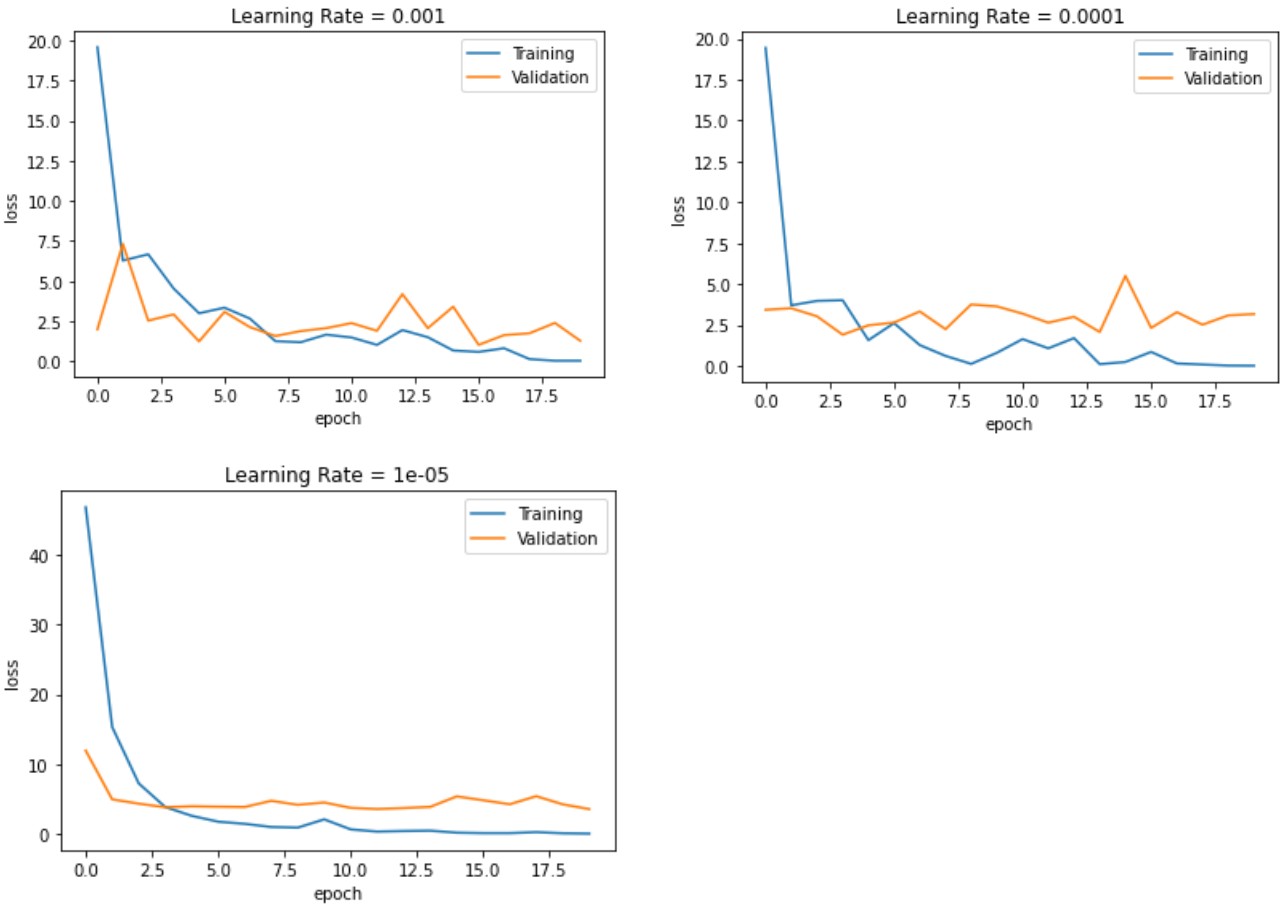


Figure 14: comparison of different learning rates

After experimenting with 3 different values, we found that setting the learning rate to 0.00001 gave us the best result in terms of convergence (no fluctuating) and accuracy.

Also, another approach we tried is setting a dynamic learning rate that changes from high to low after each epoch.

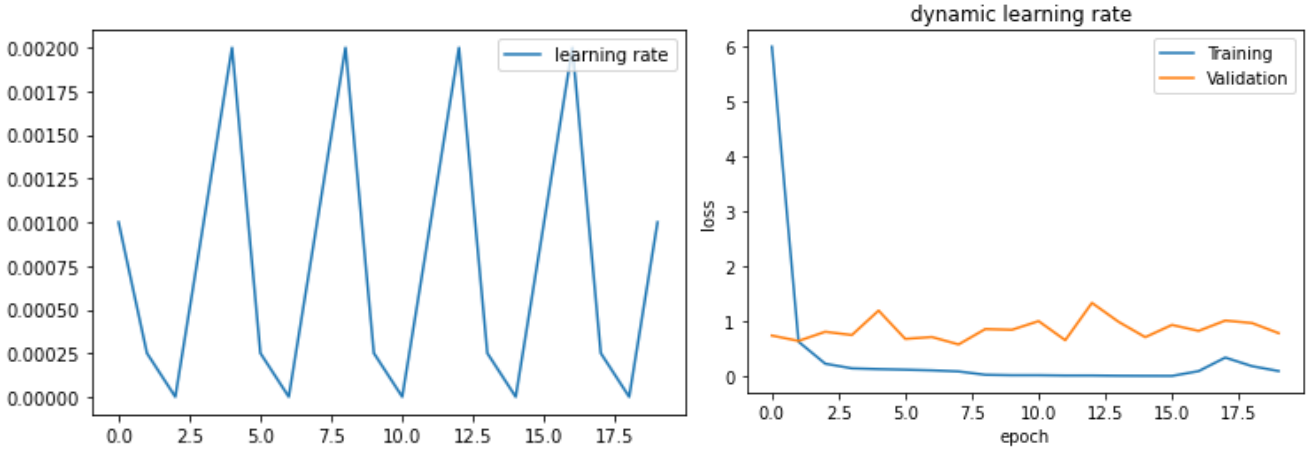


Figure 15: dynamic learning rate and its effect

This approach gave us almost similar results to setting the learning 0.00001, but it converges faster.

Strategy 2: Changing the Batch size

Computing the gradient of loss functions or the optimization algorithm is a very expensive operation. It makes no sense to do it after each single image. This is where the batch size is used. It's basically adding the loss of a certain number of images together, and then the model computes and updates the weights. For example, if we have 1000 images and a batch size value of 100, we will be updating the weights 10 times in each epoch.

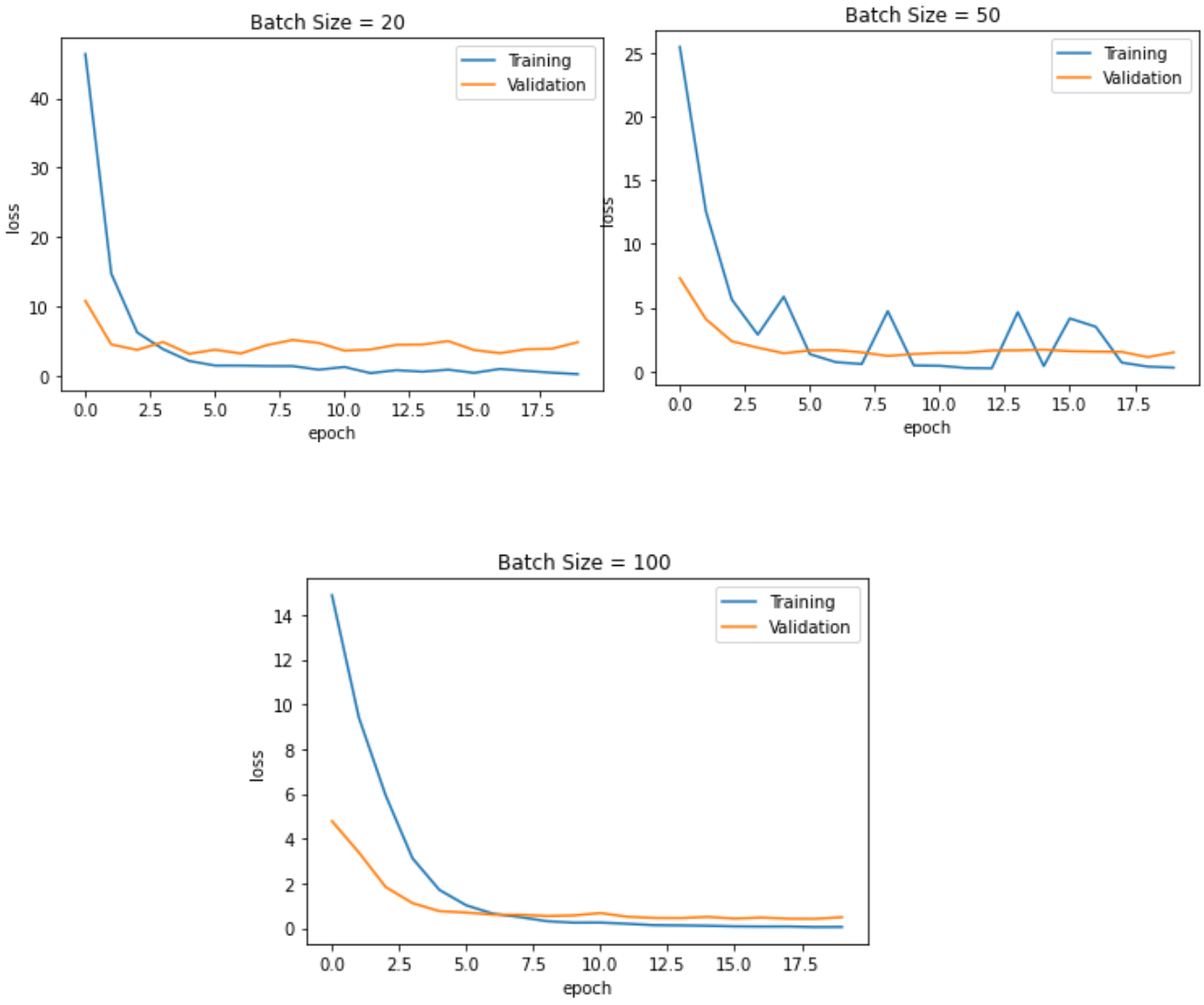


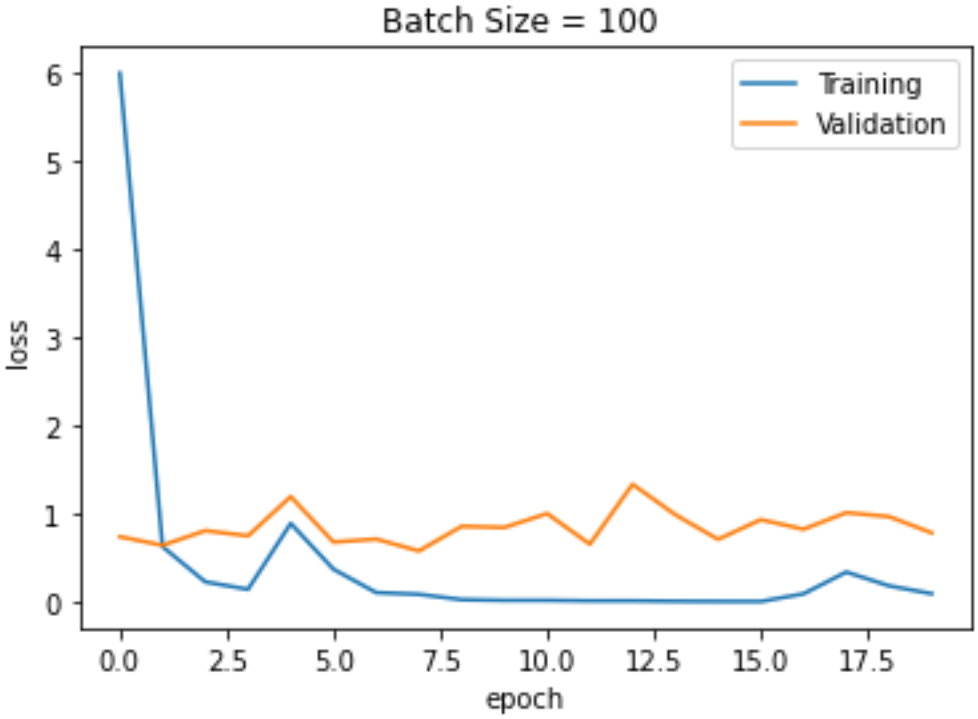
Figure 16: different batch sizes and its effect on the training process

We can see that a lower batch size like 20 and 50 compared to a value of 100 will result in a lot of fluctuating and bad validation results. This is due to the fact that the weights will be updating too many times that it might lead to overfitting.

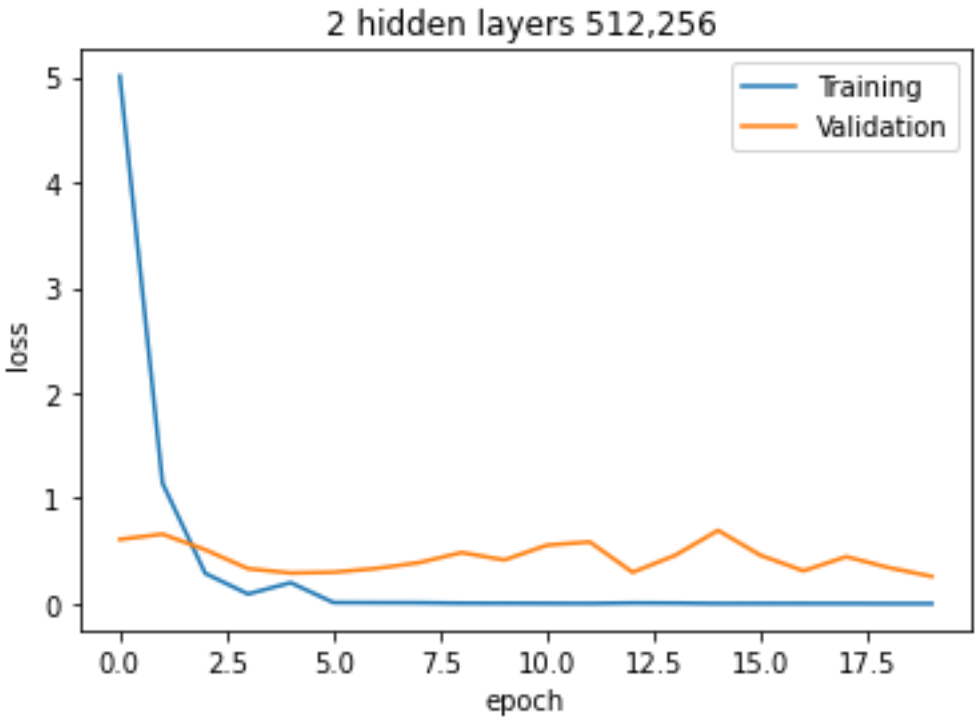
Strategy 3: changing the number of hidden layers and neurons

Like we said before, there is no rule to set a fixed number of hidden layers and neurons. So, we decided to experiment with 3 different setups:

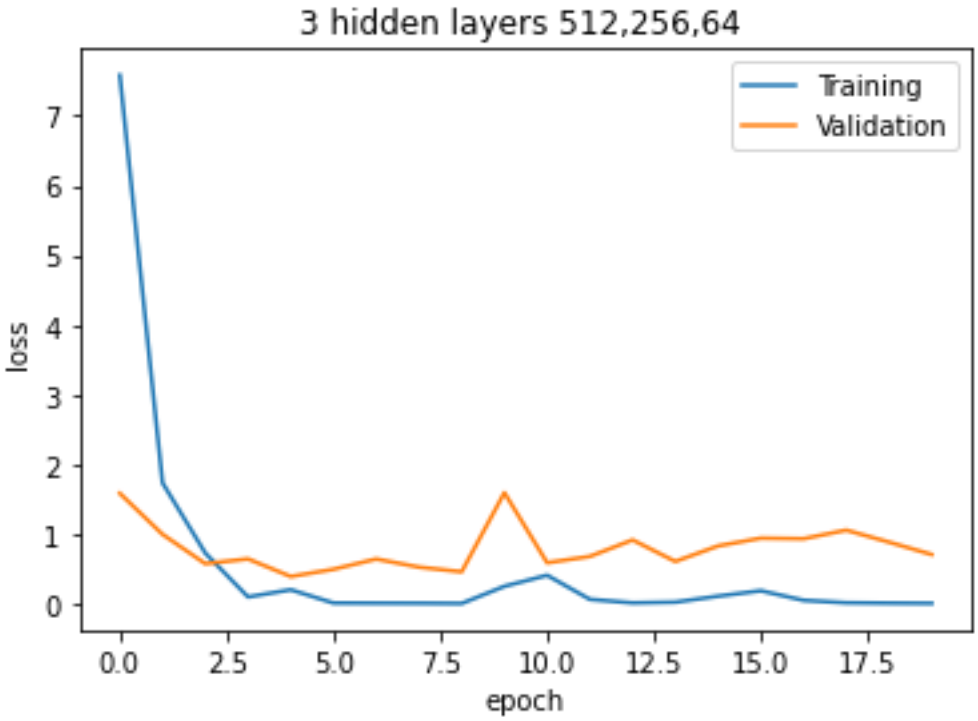
- 1 hidden layer with 512 neurons:



- 2 hidden layers with 512 and 256 neurons



- 3 hidden layers with 512,256,64 neurons



Evaluating the result

After all the changes, we, finally, reached to a perfect model that has 2 hidden layers of 512 and 256 neurons, trained it with a learning rate of 0.00001, and with a batch size of 100.

To represent the results of the achieved model, we need to first understand what a confusion matrix means.

The confusion matrix is a tool to analyze the perferomance of a model in a more specific way rather than just looking to the accuracy of the model.

A 2 class confusion matrix contains :

- True Positive : a positive target class that was classifed by the model positive
- True Negative : a negative target class that was classifed by the model negative
- False Positive : a negative target class that was classifed by the model positive
- False Negative : a positive target class that was classifed by the model negative

		Prediction	
		positive	negative
Target	positive	*TP*	*FN*
	negative	*FP*	*TN*

Figure 17: General structure of the confusion matrix

In our case, a positive case is when a person is wearing a mask, otherwhise, it's a negative case. The resulted confusion matrix on 1136 test cases gave the following matrix :

Confusion Matrix		Prediction	
		Positive	**Negative**
Target	**Positive**	567	7
	Negative	2	560

We can extract 2 important values from this matrix like :

- The classification accuracy : $\frac{TP+TN}{TP+TN+FP+FN} = \frac{1127}{1136} = 0.99$
- The misclassification rate : $\frac{FP+FN}{TP+TN+FP+FN} = \frac{9}{1136} = 0.007$

But the most crucial thing about this matrix is that the FP number is low. It's important to have a low value of False Positives because it's not tolerable to classify a person not wearing a mask as someone wearing a mask. But, a person wearing a mask and classifed to be not wearing one (FN) is considered normal and not a risk.

General Conclusion and Perspectives

Our project entitled “Covid-19 mask detection” is about developing a solution for detecting if people are wearing masks or not. This project can help in tracking if people are following government regulations during the pandemic, or maybe it can help in access control into certain building for people who doesn’t follow the safety procedures.

Throughout this document, we presented the workflow of our project by defining each the goal of each step, and the technical details within it. Next, we started experimenting with different values of some parameters to find a decent solution which we finally did.

For future improvements, our first objective is to improve the face detection module of our solution because the current face detection we are using is not very efficient in terms of finding all the faces in a picture. If we can’t improve the face detection algorithm to the desired results, we can try to tackle the project from another approach. We can try “detecting facial masks” instead of “classifying if a person is wearing a mask or not”. This is a completely different approach from our work, and it can be a whole project by itself.

Finally, we personally appreciated working on this project. It was a very enriching technical experience as far as it was a good practice to work on new technologies we never used before. It allowed us to set in practice some of the knowledge acquired during the two academic years in SUP’COM. And to end, it was an excellent opportunity to develop our observation, analysis, design, and programming skills with a good introductive experience in the machine learning field.

Bibliography and Webography

Dipanjan Sarkar, Raghav Bali, Tamoghna Ghosh. Book, “Hands-On Transfer Learning with Python”, 2018

Ahmed Fawzy Gad. Book, “Practical Computer Vision Applications Using Deep Learning with CNNs”, 2018

Ian Pointer. Book, “Programming PyTorch for Deep Learning”, 2019

Jeremy Howard, Research Paper, “Face Mask against COVID-19: An Evidence Review”, 2020

https://www.pyimagesearch.com/2018/02/26/face-detection-with-opencv-and-deep-learning
(Consulted on 15/05/2020)

https://machinelearningmastery.com/understand-the-dynamics-of-learning-rate-on-deep-learning-neural-networks/
(Consulted on 02/05/2020)

https://www.kaggle.com/caglaruslu/real-time-medical-mask-detection
(Consulted on 28/04/2020)